Ollama Tool Calling: Integrate AI with ANY Python Function in 7 mins!

Mervin Praison
75.3K subscribers
Join Subscribe

624 Share Ask Download

16,863 views Nov 27, 2024 #Ollama #AI #Python
Master Ollama Function Calling & Custom Tools Integration
In this comprehensive guide, learn how to leverage Ollama's new native function calling capabilities to create powerful AI applications. From basic setup to advanced implementations, we cover everything you need to know!

What You'll Learn:
Create custom tools and integrate them with Ollama
Implement async functionality for better performance
Utilize existing Python functions as tools
Real-world examples with stock prices and web scraping

https://mer.vin/2024/11/ollama-tools-...

Tutorial Breakdown:
Creating Custom Tools
Installing required packages (Ollama, Yahoo Finance)
Setting up basic function structure
Integration with stock price API
Ollama Integration
Function calling implementation
Tool definition and setup
Argument parsing and execution
Async Implementation
Converting synchronous code to async
Performance optimization
Real-world examples

Required Packages:
Ollama
Yahoo Finance
Python Requests

Support the Channel:
Like and Subscribe for more AI tutorials
Share with fellow developers
Enable notifications to stay updated

Keywords:
#Ollama #AI #Programming #Python #Tutorial #FunctionCalling #WebDevelopmer

Timestamp:
0:00 - Introduction to Ollama Native Tools
0:26 - Why We Need Tools for LLMs
0:59 - Tutorial Overview & Channel Info
1:35 - Step 1: Creating Custom Tool
2:29 - Step 2: Integrating with Ollama
4:22 - Step 3: Using Async Functions
5:18 - Step 4: Using Existing Functions
6:53 - Conclusion

https://mer.vin/2024/11/ollama-tools-call/

OLLAMA

Ollama Tools Call

By praison November 27, 2024

Ollama Function Calling

```python
import ollama
import yfinance as yf
from typing import Dict, Any, Callable

def get_stock_price(symbol: str) -> float:
    ticker = yf.Ticker(symbol)
    price_attrs = ['regularMarketPrice', 'currentPrice', 'price']

    for attr in price_attrs:
        if attr in ticker.info and ticker.info[attr] is not None:
            return ticker.info[attr]
```

```
❯ python stock.py
Prompt: What is the current stock price of Apple?
Calling function: get_stock_price
Arguments: {'symbol': 'AAPL'}
Function output: 235.06
```

https://mer.vin/2024/11/ollama-tools-call/

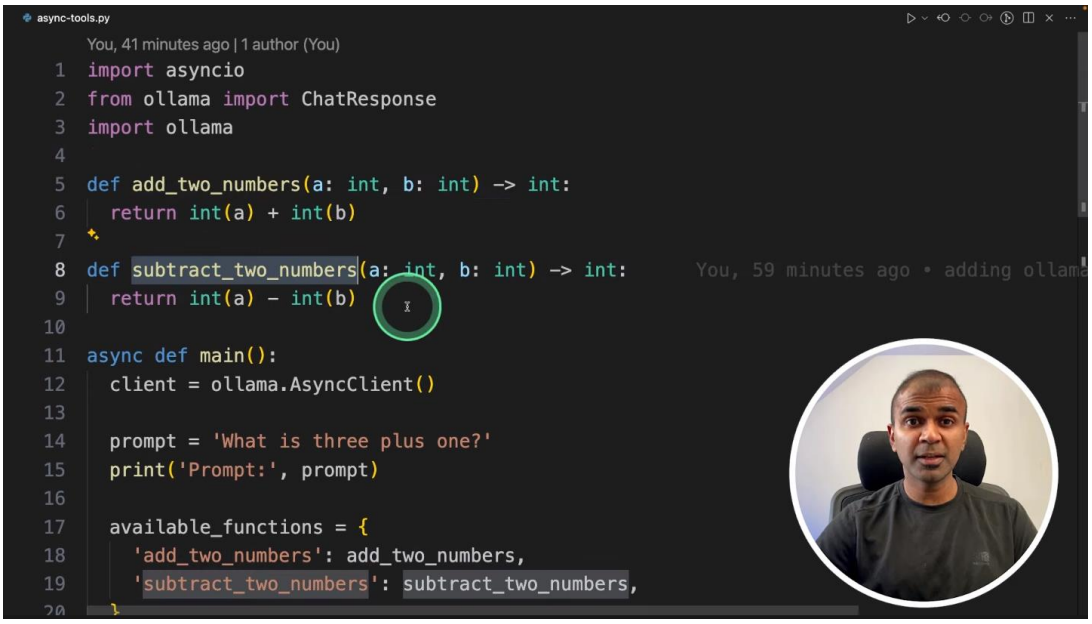
Adding Subtracting numbers

```
from ollama import chat
from ollama import ChatResponse


def add_two_numbers(a: int, b: int) -> int:
  """
  Add two numbers

  Args:
    a (int): The first number
    b (int): The second number

  Returns:
    int: The sum of the two numbers
  """
  return a + b


def subtract_two_numbers(a: int, b: int) -> int:
  """
  Subtract two numbers
  """
  return a - b
```

https://mer.vin/2024/11/ollama-tools-call/

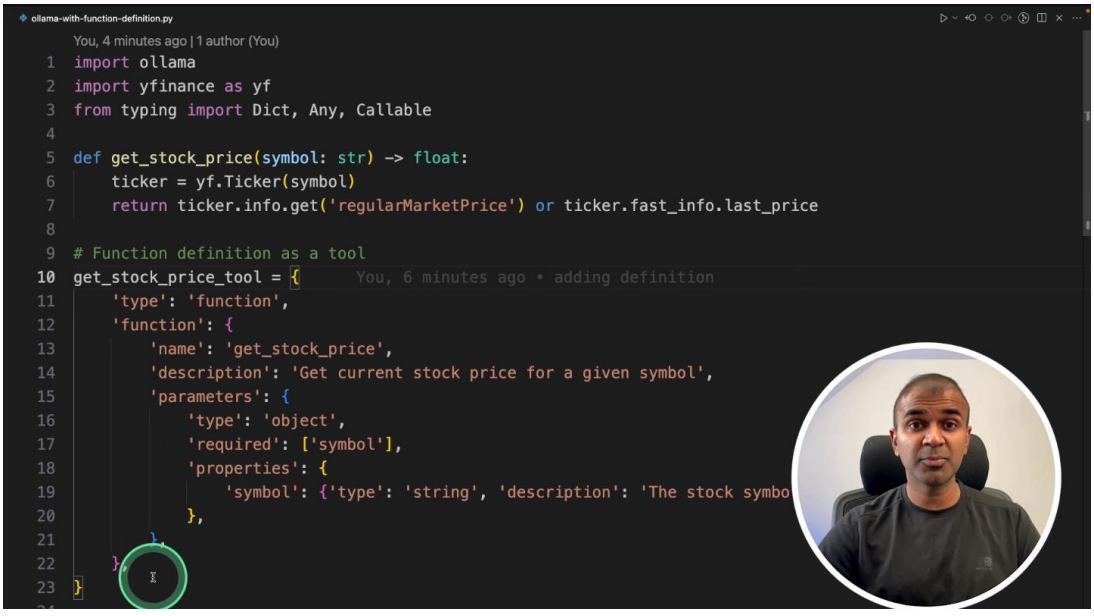
Crawl URL Ollama

```
import ollama
import requests

available_functions = {
    'request': requests.request,
}

response = ollama.chat(
    'llama3.1',
    messages=[{
        'role': 'user',
        'content': 'get the https://ollama.com webpage?',
    }],
    tools=[requests.request],
)

for tool in response.message.tool_calls or []:
    function_to_call = available_functions.get(tool.function.name)
    if function_to_call == requests.request:
        # Make an HTTP request to the URL specified in the tool call
        resp = function_to_call(
            method=tool.function.arguments.get('method'),
            url=tool.function.arguments.get('url'),
        )
        print(resp.text)
    else:
        print('Function not found:', tool.function.name)
```



Ollama Python library 0.4 with function calling improvements

November 25, 2024



Previously we will add function calling as above

But this is now all that is needed to add your own tool or function.

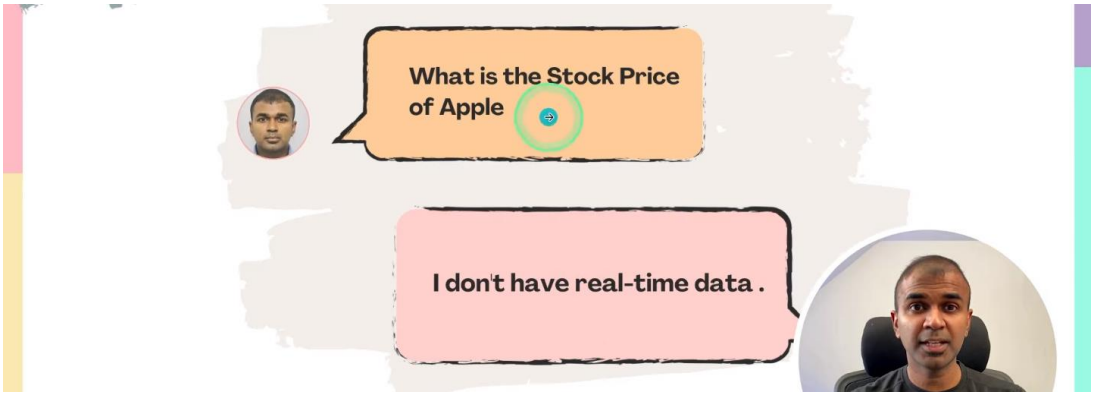
Pass the function as a tool to Ollama

Next, use the `tools` field to pass the function as a tool to Ollama:

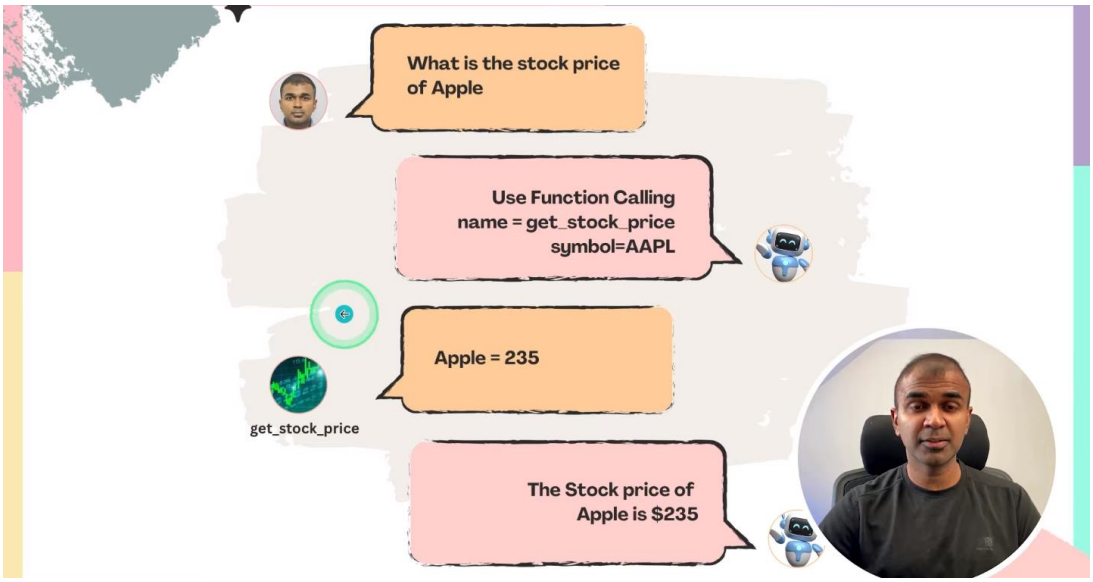
```
import ollama

response = ollama.chat(
  'llama3.1',
  messages=[{'role': 'user', 'content': 'What is 10 + 10?'}],
  tools=[add_two_numbers], # Actual function reference
)
```

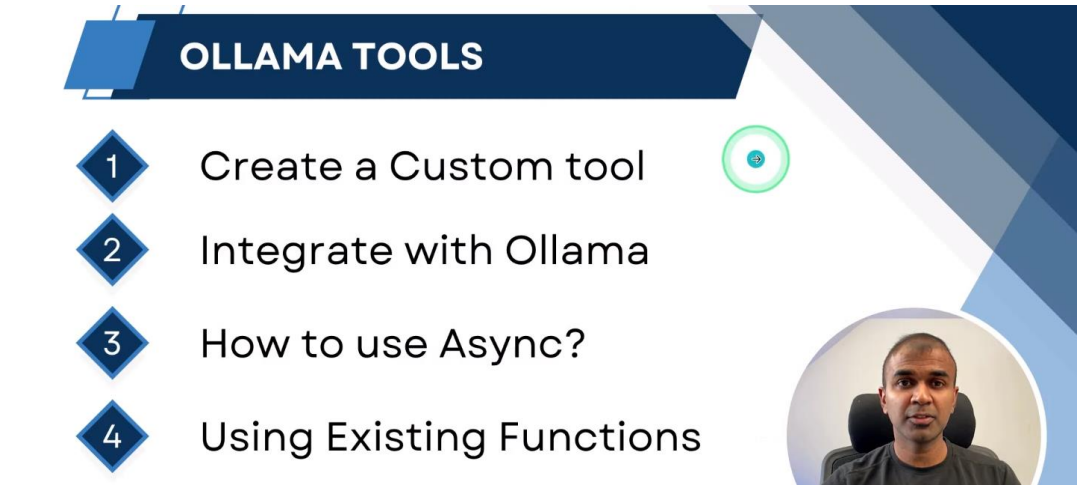
Call the function from the model response

Use the returned tool call and arguments provided by the model t
function:



Our LLMs are not very useful without tools






```
TERMINAL
main ❯ pip install -U ollama yfinance
```

```
❯ ollama pull llama3.2
pulling manifest
pulling dde5aa3fc5ff... 100%  2.0 GB
pulling 966de95ca8a6... 100%  1.4 KB
pulling fcc5a6bec9da... 100%  7.7 KB
pulling a70ff7e570d9... 100%  6.0 KB
pulling 56bb8bd477a5... 100%   96 B
pulling 34bb5ab01051... 100%  561 B
verifying sha256 digest
writing manifest
success
main ❯
```

app.py M

You, 55 seconds ago | 1 author (You)

```python
import ollama
import yfinance as yf
from typing import Dict, Any, Callable

def get_stock_price(symbol: str) -> float:
    ticker = yf.Ticker(symbol)
    return ticker.info.get('regularMarketPrice') or ticker.fast_info.last_price

prompt = 'What is the current stock price of Apple?'

available_functions: Dict[str, Callable] = {
    'get_stock_price': get_stock_price,
}

response = ollama.chat(
    'llama3.2',
    messages=[{'role': 'user', 'content': prompt}],
    tools=[get_stock_price],
)
```

app.py M

```python
available_functions: Dict[str, Callable] = {
}

response = ollama.chat(
    'llama3.2',
    messages=[{'role': 'user', 'content': prompt}],
    tools=[get_stock_price],
)

if response.message.tool_calls:
    for tool in response.message.tool_calls:
        if function_to_call := available_functions.get(tool.function.name):
            print('Calling function:', tool.function.name)
            print('Arguments:', tool.function.arguments)
            print('Function output:', function_to_call(**tool.function.arguments))
        else:
            print('Function', tool.function.name, 'not found')
```

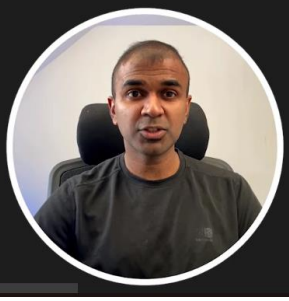
```
❯ python app.py
Calling function: get_stock_price
Arguments: {'symbol': 'AAPL'}
Function output: 235.05999755859375
main !1 ❯
```

3 How to use Async?

async-tools.py

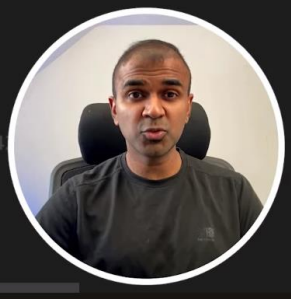
You, 41 minutes ago | 1 author (You)

```python
import asyncio
from ollama import ChatResponse
import ollama

def add_two_numbers(a: int, b: int) -> int:
  return int(a) + int(b)

def subtract_two_numbers(a: int, b: int) -> int:
  return int(a) - int(b)

async def main():
  client = ollama.AsyncClient()

  prompt = 'What is three plus one?'
  print('Prompt:', prompt)

  available_functions = {
    'add_two_numbers': add_two_numbers,
    'subtract_two_numbers': subtract_two_numbers,
  }
```



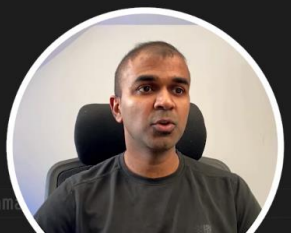
async-tools.py

```python
async def main():
  client = ollama.AsyncClient()

  prompt = 'What is three plus one?'
  print('Prompt:', prompt)

  available_functions = {
    'add_two_numbers': add_two_numbers,
    'subtract_two_numbers': subtract_two_numbers,
  }

  response: ChatResponse = await client.chat(
    'llama3.1',
    messages=[{'role': 'user', 'content': prompt}],
    tools=[add_two_numbers, subtract_two_numbers],
  )

  if response.message.tool_calls:
    # There may be multiple tool calls in the response
    for tool in response.message.tool_calls:
```



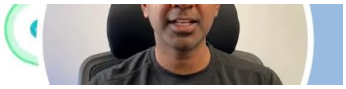
async-tools.py

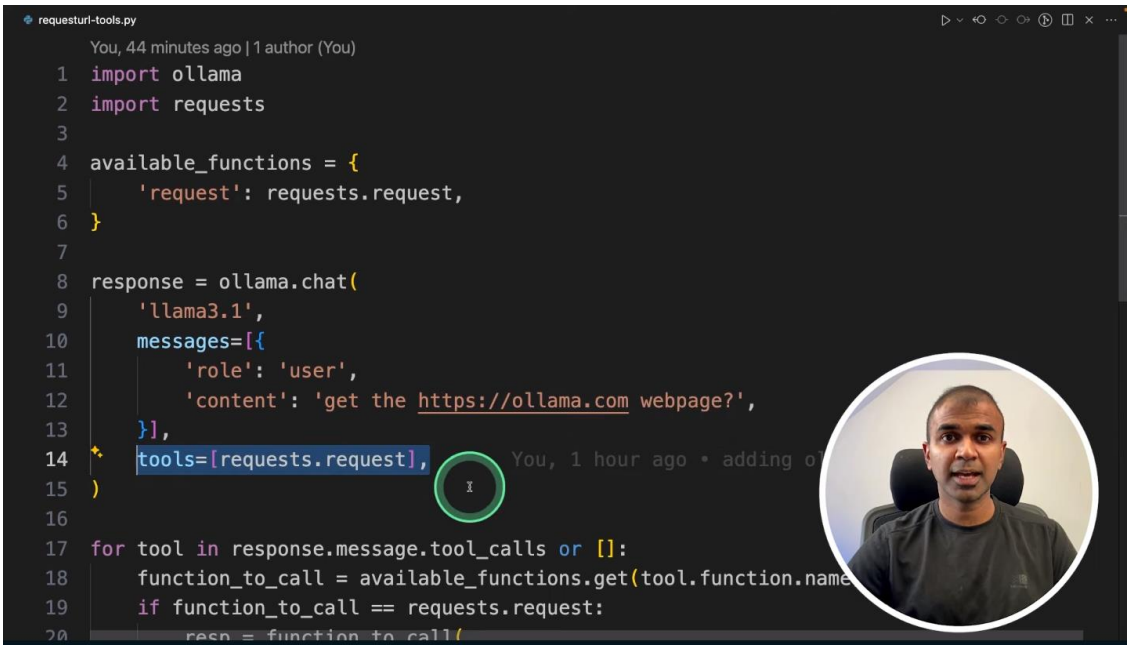
```python
async def main():
  if response.message.tool_calls:
    # There may be multiple tool calls in the response
    for tool in response.message.tool_calls:
      # Ensure the function is available, and then call it
      if function_to_call := available_functions.get(tool.function.name):
        print('Calling function:', tool.function.name)
        print('Arguments:', tool.function.arguments)
        print('Function output:', function_to_call(**tool.function.arguments))
      else:
        print('Function', tool.function.name, 'not found')


if __name__ == '__main__':
  try:
    asyncio.run(main())
  except KeyboardInterrupt:
    print('\nGoodbye!')
```



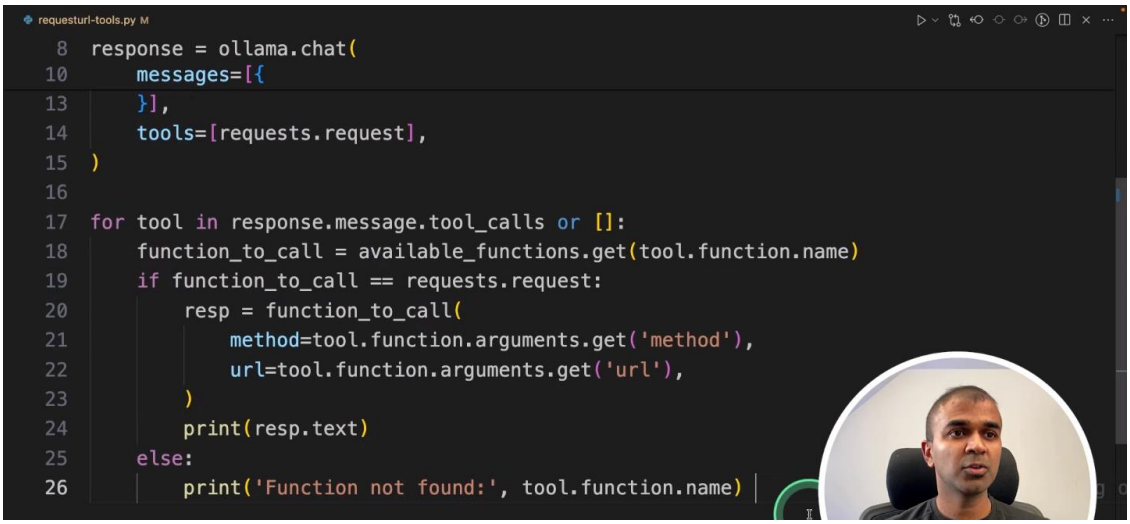
```
❯ python async-tools.py
Prompt: What is three plus one?
Calling function: add_two_numbers
Arguments: {'a': 1, 'b': 3}
Function output: 4
main !1 ❯
```

Using Existing Functions





We are going to be using the **requests.request** package that should go and scrape that URL webpage



```
<pre class="wp-block-code"><code>import gradio as gr
from transformers import pipeline
import numpy as np

transcriber = pipeline("automatic-speech-recognition", model="openai/whisper-base.en")

def transcribe(stream, new_chunk):
    if new_chunk is None:
        return None, ""

    sr, y = new_chunk

    # Convert to mono if stereo
    if y.ndim &gt; 1:
        y = y.mean(axis=1)

    y = y.astype(np.float32)
    y /= np.max(np.abs(y)) + 1e-7  # Safe normalization

    # Initialize or update stream
    if stream is not None:
        stream = np.concatenate(&#91;stream, y])
    else:
        stream = y

    # Keep only last 5 seconds to prevent memory issues
```

```
<script src="https://mer.vin/wp-includes/js/dist/vendor/wp-polyfill.min.js?ver=3.15.0" id="wp-pol
yfill-js"></script>
<script id="wpcf7-recaptcha-js-extra">
var wpcf7_recaptcha = {"sitekey":"6LeXjVkpAAAAADsBX0MlSwbKS2RIu9FqYv2RNUzf","actions":{"homepage"
:"homepage","contactform":"contactform"}};
</script>
<script src="https://mer.vin/wp-content/plugins/contact-form-7/modules/recaptcha/index.js?ver=5.9
.8" id="wpcf7-recaptcha-js"></script>
<script src="https://stats.wp.com/e-202448.js" id="jetpack-stats-js" data-wp-strategy="defer"></s
cript>
<script id="jetpack-stats-js-after">
_stq = window._stq || [];
_stq.push([ "view", JSON.parse("{\"v\":\"ext\",\"blog\":\"47859466\",\"post\":\"0\",\"tz\":\"0\",
\"srv\":\"mer.vin\",\"j\":\"1:13.9.1\"}") ]);
_stq.push([ "clickTrackerInit", "47859466", "0" ]);
</script>

        </body>
</html>
<!--
Performance optimized by Redis Object Cache. Learn more: https://wpredis

Retrieved 15329 objects (671 KB) from Redis using PhpRedis (v6.1.0).
-->

main !2 ❯
```